

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BTL Phát triển ứng dụng trên thiết bị di động

**Tinner: Nền tảng di động tối ưu hóa
quá trình đặt món và địa điểm**

Giảng viên: Ths. Hoàng Lê Hải Thanh
Lớp: L01
Khoa: Khoa học và Kỹ thuật Máy tính

TP. HỒ CHÍ MINH, THÁNG 05/2026

Danh sách thành viên và Phân công

Dưới đây là danh sách các thành viên tham gia thực hiện bài tập lớn và bảng phân công chi tiết:

MSSV	Thành viên	Phân công
2313063	Phạm Ngọc Tây	[Nhiệm vụ chi tiết]
[MSSV]	Âu Nguyễn Hùng Mạnh	[Nhiệm vụ chi tiết]
[MSSV]	Nguyễn Tiến Nam	[Nhiệm vụ chi tiết]
[MSSV]	Phan Đình Khang	[Nhiệm vụ chi tiết]
[MSSV]	Thanh Nhân	[Nhiệm vụ chi tiết]

Bảng 1: Bảng phân công nhóm

Mục lục

Danh sách thành viên và Phân công	1
1 Giới thiệu	3
1.1 Đặt vấn đề	3
1.2 Mục tiêu và Mục đích	3
1.3 Nhu cầu thực tế và Tác động tiềm năng	4
2 Tiếp cận và Phương pháp luận	4
2.1 Quy trình phát triển	4
2.2 Công cụ cộng tác	4
2.3 Cơ sở cho việc ra quyết định	4
3 Tính năng và Chức năng của ứng dụng	5
3.1 Các tính năng cốt lõi	5
3.2 Các chức năng nâng cao	5
4 Chi tiết triển khai	6
4.1 Tổ chức mã nguồn	6
4.2 Backend triển khai tại <code>apps/backend</code>	6
4.2.1 Luồng khởi động và middleware chung	6
4.2.2 Xác thực và phân quyền	7
4.2.3 Các module nghiệp vụ	7
4.2.4 Tầng gọi ý và tích hợp Overpass	7
4.2.5 Health check	8
4.3 Ứng dụng di động triển khai tại <code>apps/mobile</code>	8
4.4 Giao diện web tham chiếu tại <code>frontend</code>	8
4.5 Phát triển cục bộ và kiểm thử	9
4.5.1 Chạy hệ thống cục bộ bằng Docker Compose	9
4.5.2 Quy trình migration	9
4.5.3 Kiểm thử backend	9
5 Trải nghiệm người dùng	9
5.1 Luồng người dùng chi tiết	9
5.1.1 Tính năng 1: Vuốt để khám phá và lưu món ăn	9
5.1.2 Tính năng 2: Quản lý danh sách “Muốn thử”	10
5.2 Ảnh chụp màn hình	10
6 Sơ đồ kiến trúc hệ thống	13
6.1 Mô tả kiến trúc tổng quan	13
6.2 Sơ đồ	13
6.3 Luồng yêu cầu gọi ý nhà hàng	14
7 Thiết kế dịch vụ web	14
7.1 Kiểu dịch vụ và quy ước định tuyến	14
7.2 Xác thực và Phân quyền	15
7.3 Validation, Xử lý lỗi và Bảo mật	15
7.4 Tài liệu hóa API	16

8	Chiến lược triển khai	16
8.1	Mục tiêu triển khai	16
8.2	Công nghệ triển khai	17
8.3	Triển khai Backend trên Render	17
8.4	Chiến lược cơ sở dữ liệu trên Aiven	18
8.5	Triển khai Frontend trên Vercel	18
8.6	Phát hành ứng dụng di động qua EAS	18
8.7	CI/CD và Vận hành	18
8.8	Tóm tắt	19
9	Báo cáo độ bao phủ kiểm thử	19
9.1	Giới thiệu Jest và các công cụ testing	19
9.2	Cấu hình môi trường kiểm thử	19
9.3	Tiêu chuẩn độ bao phủ	20
9.4	Kết quả kiểm thử chi tiết	20
9.4.1	Module Auth (Xác thực)	21
9.4.2	Module Gợi ý món ăn	21
9.4.3	Module Tương tác người dùng	21
9.4.4	Module Bộ lọc và Món ăn	21
9.5	Các kịch bản kiểm thử API	22
9.6	Phạm vi kiểm thử E2E Layer	22
9.6.1	Phương pháp tiếp cận và Cấu trúc kiểm thử E2E	23
9.6.2	Đánh giá kết quả kiểm thử E2E	23
9.7	Gap Analysis and Code Quality	24
10	Phân tích khảo sát mức độ hài lòng và Hành vi người dùng	24
10.1	Phản hồi và Kết quả khảo sát	24
10.2	Số liệu hành vi người dùng	24
11	Đánh giá dự án và Các hướng cải thiện	24
11.1	Thành công và Thách thức	24
11.2	Các kế hoạch trong tương lai	24
12	Sản phẩm thực tế	24
12.1	Khả năng tương thích của ứng dụng	24
12.2	Tệp APK và Liên kết Google Drive	24
12.3	Tích hợp Backend và Cấu hình	24
13	Phân phối sản phẩm và Khảo sát	24
13.1	Quảng bá và Tiếp thị	24
13.2	Thiết kế và Phân tích khảo sát	24
13.3	Thu thập số liệu ứng dụng	24
13.4	Đánh giá phản hồi và Phân tích hành vi	24
13.5	Mình chứng tương tác người dùng	24

1 Giới thiệu

1.1 Đặt vấn đề

Trong cuộc sống hiện đại, đặc biệt là đối với sinh viên, nhân viên văn phòng và các cặp đôi, câu hỏi "Hôm nay ăn gì?" đã phát triển từ một câu hỏi đơn giản thành một nguồn gây căng thẳng hàng ngày. Vấn đề này biểu hiện qua ba điểm yếu (pain points) chính:

- **Sự mệt mỏi khi ra quyết định (Decision Fatigue):** Trung bình, mỗi cá nhân dành từ 15 đến 20 phút để lướt các ứng dụng giao đồ ăn một cách vô định mà không đưa ra được lựa chọn cuối cùng, thường dẫn đến việc chọn một bữa ăn ngẫu nhiên, không hài lòng hoặc bỏ bữa hoàn toàn.
- **Quá tải lựa chọn (Choice Overload):** Các nền tảng hiện nay hiển thị hàng trăm tùy chọn nhà hàng cùng lúc dưới dạng danh sách. Lượng thông tin khổng lồ này gây ra tình trạng quá tải nhận thức, khiến người dùng khó đưa ra quyết định và ít hài lòng hơn với lựa chọn cuối cùng của mình.
- **Mâu thuẫn nhóm (Social Friction):** Việc quyết định địa điểm ăn uống là một nguồn gây mâu thuẫn phổ biến cho các nhóm, thường dẫn đến sự bực bội hoặc những thỏa hiệp không thoải mái.

1.2 Mục tiêu và Mục đích

Mục tiêu chính của dự án này là phát triển **Tinner** (một từ ghép của "Tinder" và "Dinner"), một ứng dụng di động được thiết kế để giải quyết vấn đề "Tê liệt phân tích" (Analysis Paralysis) trong việc lựa chọn thực phẩm. Với khẩu hiệu "*Swipe To Bite*", Tinner đơn giản hóa quy trình ra quyết định.

Các mục đích cốt lõi của ứng dụng bao gồm:

- **Hợp lý hóa các quyết định:** Giúp người dùng chuyển từ việc mở ứng dụng sang quyết định một bữa ăn trong vòng chưa đầy 5 phút.
- **Trải nghiệm ưu tiên hình ảnh (Visual-First Experience):** Cung cấp một "Giao diện chế độ xem đơn" (Single-View Interface) hiển thị hình ảnh món ăn chất lượng cao tại một thời điểm, loại bỏ các yếu tố gây xao nhãng và kích thích sự thèm ăn.
- **Khám phá thông qua Gamification:** Triển khai cơ chế quẹt thẻ (Vuốt sang trái để bỏ qua, Vuốt sang phải để lưu/thích) giúp biến một công việc tẻ nhạt thành một hoạt động hấp dẫn.
- **Truy cập thông tin liền mạch:** Khi thích một món ăn, ứng dụng ngay lập tức hiển thị Thẻ thông tin nhà hàng với các chi tiết quan trọng (địa chỉ, giờ mở cửa, tích hợp bản đồ và đánh giá) để hoàn tất quyết định.
- **Quản lý hành động:** Cho phép người dùng duy trì danh sách "Muốn thử" cá nhân để xem xét, so sánh và điều hướng đến các nhà hàng đã chọn một cách hiệu quả.

1.3 Nhu cầu thực tế và Tác động tiềm năng

Nghiên cứu thị trường toàn diện chỉ ra một khoảng trống đáng kể trong hệ sinh thái ứng dụng liên quan đến thực phẩm hiện nay. Trong khi các siêu ứng dụng giao hàng vượt trội về hậu cần, các nền tảng mạng xã hội vượt trội về cảm hứng hình ảnh và dịch vụ bản đồ vượt trội về dữ liệu vị trí, thì không có mô hình lai (hybrid model) ưu việt nào được tối ưu hóa thuần túy cho việc khám phá nhà hàng nhanh chóng, thú vị và có cấu trúc.

Tinner giải quyết nhu cầu thực tế này bằng cách định vị mình là một "Công cụ ra quyết định" (Decision Engine) hơn là một trung gian giao dịch. Bằng cách tập trung vào giai đoạn khám phá và kết hợp các tính năng như Bộ lọc thông minh (Smart Filters) và Bộ sưu tập cá nhân (Personal Collections), ứng dụng giúp giảm tải nhận thức và thu hẹp khoảng cách giữa thông tin có cấu trúc và trải nghiệm người dùng hấp dẫn.

Tác động tiềm năng của Tinner là rất lớn, đặc biệt là đối với Gen Z và Millennials. Nó hứa hẹn sẽ giảm đáng kể thời gian lãng phí cho các quyết định ăn uống hàng ngày, giảm bớt mâu thuẫn xã hội liên quan đến lựa chọn của nhóm và cung cấp một giải pháp trực quan và thú vị cho một vấn đề nan giải hàng ngày.

2 Tiếp cận và Phương pháp luận

2.1 Quy trình phát triển

Để đảm bảo cung cấp nhanh chóng Sản phẩm khả thi tối thiểu (Minimum Viable Product - MVP) và cho phép cải tiến lặp đi lặp lại, dự án đã áp dụng phương pháp luận phát triển Agile. Cách tiếp cận này cho phép nhóm ưu tiên các tính năng cốt lõi—cụ thể là các chức năng khám phá và ra quyết định—trong khi vẫn duy trì tính linh hoạt để kết hợp phản hồi của người dùng. Chu kỳ phát triển được chia thành các giai đoạn (sprints) tập trung, cho phép thử nghiệm liên tục, tinh chỉnh giao diện người dùng và xác nhận triết lý thiết kế "Ưu tiên hình ảnh" (Visual-First).

2.2 Công cụ cộng tác

Việc giao tiếp hiệu quả và kiểm soát phiên bản (Version Control) được duy trì bằng cách sử dụng các công cụ cộng tác tiêu chuẩn công nghiệp. Git và GitHub được sử dụng để quản lý mã nguồn, cho phép nhóm xử lý việc phát triển tính năng song song, thực hiện đánh giá mã (Code Review) và theo dõi các vấn đề (Issues) một cách liền mạch. Các tài liệu như luồng người dùng (User Flows), nghiên cứu thị trường và ma trận tính năng được duy trì theo định dạng Markdown trong kho lưu trữ để đảm bảo tất cả các thành viên trong nhóm đều có quyền truy cập vào một nguồn thông tin duy nhất (Single Source of Truth).

2.3 Cơ sở cho việc ra quyết định

Việc lựa chọn tính năng và kiến trúc thiết kế được thúc đẩy bởi mục tiêu giải quyết các điểm yếu (pain points) cụ thể của người dùng, như đã nêu trong Cơ sở lý luận sản phẩm (Product Rationale):

- **Tập trung vào các giá trị cốt lõi (Khám phá & Quyết định):** Ưu tiên không phải là xây dựng một danh bạ tĩnh, mà là một công cụ động trực tiếp chấm dứt sự thiếu quyết đoán của người dùng. Các tính năng được đánh giá dựa trên Ma trận

Tác động và Nỗ lực (Impact vs. Effort Matrix), ưu tiên các chức năng có tác động cao như Thẻ thông tin (Info Card) và Bộ sưu tập cá nhân (Personal Collections).

- **Triết lý "Ưu tiên hình ảnh" (Visual-First):** Hoạt động trên nguyên tắc "người ta ăn bằng mắt", thiết kế ưu tiên mạnh mẽ hình ảnh thực phẩm rực rỡ hơn là các menu dày đặc văn bản. Điều này tạo niềm tin và khơi gợi cảm xúc ngay từ cái nhìn đầu tiên.
- **Giải quyết nghịch lý của sự lựa chọn (Paradox of Choice):** Bằng cách chỉ hiển thị một món ăn tại một thời điểm và sử dụng cơ chế quẹt thẻ (Gamified swipe mechanism), ứng dụng cố tình giới hạn các tùy chọn đồng thời để giảm tải nhận thức và biến việc ra quyết định thành một trải nghiệm giải trí.
- **Chiến lược duy trì người dùng (Retention Strategy):** Danh sách "Muốn thử" (Bộ sưu tập cá nhân) được triển khai như một "lưỡi câu" chính (primary hook). Khi người dùng bỏ công sức để lưu các địa điểm yêu thích của họ, họ tạo ra một cơ sở dữ liệu cá nhân có giá trị, làm tăng đáng kể Tỷ lệ duy trì (Retention Rate) của ứng dụng.

3 Tính năng và Chức năng của ứng dụng

3.1 Các tính năng cốt lõi

Sản phẩm khả thi tối thiểu (MVP) tập trung vào việc cung cấp luồng người dùng thiết yếu: khám phá các món ăn, lưu các sở thích, xem lại các lựa chọn và điều hướng đến điểm đến.

- **Vuốt để khám phá (Cơ chế Swipe-to-Match):** Đây là cơ chế khám phá chính. Hệ thống lấy các món ăn gần đó dựa trên vị trí của người dùng và hiển thị chúng trong Giao diện Chế độ xem Đơn (Single-View Interface). Người dùng vuốt sang trái để bỏ qua hoặc vuốt sang phải để lưu món ăn. Cơ chế này thúc đẩy việc ra quyết định nhanh chóng và trực quan.
- **Quản lý danh sách "Muốn thử" (Bộ sưu tập cá nhân):** Một không gian dành riêng nơi người dùng có thể xem lại và quản lý các món ăn họ đã lưu trước đó thông qua việc vuốt. Nó cho phép người dùng so sánh các lựa chọn, cân nhắc lại các lựa chọn và xóa các mục, chuyển đổi giai đoạn khám phá thành một quy trình ra quyết định có thể hành động.
- **Thẻ thông tin nhà hàng (Restaurant Info Card):** Khi tương tác với một món ăn, ứng dụng sẽ hiển thị một thẻ toàn diện bao gồm hình ảnh thực phẩm chất lượng cao, tóm tắt thực đơn, giờ mở cửa và các đánh giá nổi bật. Điều này cung cấp một cái nhìn tổng quan nhanh chóng mà không yêu cầu chuyển đổi nhiều màn hình.

3.2 Các chức năng nâng cao

Để nâng cao trải nghiệm người dùng ngoài việc khám phá cơ bản, một số chức năng nâng cao đã được tích hợp vào nền tảng:

- **Bộ lọc thông minh (Smart Filters):** Người dùng có thể thu hẹp kết quả tìm kiếm một cách chính xác dựa trên nhu cầu thời gian thực bằng cách lọc theo bán kính (khoảng cách), mức giá, loại ẩm thực và xếp hạng.
- **Tích hợp Bản đồ & Điều hướng:** Người dùng có thể xem các nhà hàng đã thích trên bản đồ hoặc điều hướng trực tiếp đến vị trí bằng một lần chạm thông qua việc tích hợp Google Maps hoặc Apple Maps, giúp tối ưu hóa lộ trình di chuyển.
- **Tích hợp API bên thứ ba:** Để tăng uy tín dữ liệu và cung cấp cái nhìn khách quan, ứng dụng tích hợp với các nguồn bên ngoài (ví dụ: Google Places, Yelp) để hiển thị điểm số và đánh giá đã được xác minh.
- **Khớp nhóm và Gamification (Group Match and Gamification):** Các tính năng như "Khớp nhóm" (khớp chỉ xảy ra khi mọi người trong nhóm cùng vượt sang phải) và "Vòng quay may mắn" được thiết kế để giải quyết các xung đột ăn uống trong nhóm và biến việc chọn nhà hàng thành một trò chơi xã hội thú vị.
- **Tour ẩm thực (Food Tours):** Ứng dụng hỗ trợ các hành trình được tuyển chọn để biến một bữa tối tiêu chuẩn thành một cuộc phiêu lưu nhỏ, kích thích sự tò mò và tăng thời gian người dùng tương tác với nền tảng.

4 Chi tiết triển khai

4.1 Tổ chức mã nguồn

Dự án tổ chức theo mô hình monorepo, sử dụng `pnpm` làm trình quản lý gói/workspace, yêu cầu Node.js 22+. Các thành phần chính:

- **apps/backend:** Backend REST API xây dựng bằng NestJS 11, dùng Prisma 7 + PostgreSQL.
- **apps/mobile:** Ứng dụng di động Expo (React Native + TypeScript). Phần native chính nằm trong `apps/mobile/app-native` (entry: `AppShell.tsx`, các screen `Login/Signup/Swipe/Map/Collections/Filters`).
- **packages/types:** DTO/enum dùng chung giữa backend và client.
- **frontend:** Giao diện web tham chiếu (Vite + React), không bắt buộc cho luồng mobile chính.

4.2 Backend triển khai tại apps/backend

Backend dùng TypeScript với framework NestJS 11. Truy cập dữ liệu qua Prisma 7 kết nối PostgreSQL (Aiven trên production, Docker Postgres trên local).

4.2.1 Luồng khởi động và middleware chung

Trong `main.ts`, ứng dụng bật CORS với `credentials`, dùng `cookie-parser`, đăng ký:

- Một global filter `GlobalExceptionHandler (@Catch())` bắt mọi exception — phân biệt `HttpException` (trả status/message đã chuẩn hoá) và các lỗi không lường trước (log full server-side, trả 500 `Internal server error`). Ở môi trường non-production, response thêm trường `details` chứa danh sách lỗi validation; ở production thì giữ message chung để không lộ chi tiết.
- Một global `ValidationPipe` với `transform`, `whitelist`, `forbidNonWhitelisted`.
- Swagger UI tại `/api/docs` (xem mục 7.4).

4.2.2 Xác thực và phân quyền

Hệ thống dùng JWT (`@nestjs/jwt`, `passport-jwt`) với 2 token:

- `accessToken` (TTL 15 phút, mặc định) — gửi trong header `Authorization: Bearer`.
- `refreshToken` (TTL 30 ngày, mặc định) — lưu hash trong bảng `refresh_tokens`, dùng để rotate access token.

`JwtAuthGuard` đăng ký toàn cục qua `APP_GUARD` trong `AppModule`; các endpoint công khai đánh dấu bằng decorator `@Public()` (ví dụ `/auth/register`, `/auth/login`, `/auth/refresh`, `/health`).

Luồng đăng ký (POST `/api/v1/auth/register`) yêu cầu `username` (3–50 ký tự, chữ/số/underscore), `email`, `password` (8–128, có chữ và số), `confirmPassword` và `fullName` tùy chọn. Sau khi tạo user, hệ thống đồng thời khởi tạo bản ghi `user_filters` mặc định để các luồng *suggestions* sẵn sàng dùng ngay.

4.2.3 Các module nghiệp vụ

- `AuthModule`: `register/login/refresh/logout/me`.
- `DishesModule`: trả danh sách món ăn (16 dish enum: phở, bún bò, cơm tấm, bánh mì...).
- `FiltersModule`: lưu `maxDistanceKm`, `minRating`, `cuisines`, `dietary`, `priceRanges` cho từng user.
- `InteractionsModule`: ghi nhận hành vi swipe-right (`SAVE_RESTAURANT`) và like-dish (`LIKE_DISH`).
- `SuggestionsModule`: cốt lõi của trải nghiệm swipe — chi tiết ở 4.2.4.
- `HealthController`: endpoint kiểm tra trạng thái (xem 4.2.5).

4.2.4 Tăng gợi ý và tích hợp Overpass

`SuggestionsService` dùng chiến lược *DB-first* kết hợp Overpass API (OpenStreetMap) làm nguồn dữ liệu:

1. Với mỗi request GET `/api/v1/suggestions?lat&lng[&dishType]`, service đếm số nhà hàng đã đồng bộ gần đây trong bounding box quanh user (cột `lastSyncedAt` ≤ 7 ngày). Nếu đạt ngưỡng (mặc định 20 quán) thì dùng luôn dữ liệu DB.

2. Khi vùng chưa fresh, gọi 1 query Overpass API trong bán kính 7 km, filter amenity in {restaurant, fast_food, cafe, food_court}, sau đó bulk insert qua `prisma.restaurant.createMany({ skipDuplicates: true })`.
3. Với mỗi POI, service phân loại dishTypes bằng helper `detectDishTypesInText` dựa trên tag `name + cuisine`. POI không có dishType nào (vd nhà hàng generic không nêu món) vẫn được lưu nhưng bị loại khỏi deck swipe (filter `dishTypes: { isEmpty: false }`).
4. Cuối cùng, đọc DB lọc theo bbox + Haversine distance, shuffle ngẫu nhiên cho chế độ mixed (không dishType) hoặc filter chính xác `dishTypes.has(dishType)` cho chế độ single-dish.

Cách tiếp cận này giảm độ trễ lần đầu cho một vùng từ vài phút (N+1 upsert tuần tự) xuống dưới 10 giây, và các request sau cùng vùng chỉ tốn ~300 ms vì hoàn toàn phục vụ từ DB.

4.2.5 Health check

HealthController expose GET `/api/v1/health` (đánh dấu `@Public()`), trả `{ status: "ok" }`. Render dùng endpoint này cho `healthCheckPath` trong `render.yaml`.

4.3 Ứng dụng di động triển khai tại apps/mobile

Ứng dụng mobile dùng Expo SDK 54 với React 19 và React Native. Mã native nằm trong `apps/mobile/app-native`:

- `AppShell.tsx` làm root component, quản lý state phiên đăng nhập, deck swipe và điều hướng giữa các màn hình bằng state machine (`ScreenName: login | signup | swipe | map | collections | filters`).
- Sáu screen riêng trong `screens/`: `LoginScreen`, `SignupScreen`, `SwipeScreen`, `MapScreen`, `CollectionsScreen`, `FiltersScreen`.
- Tầng API client (`api.ts`): wrapper `fetch` với JWT Bearer header, parse JSON lỗi để chỉ throw trường `message` (không hiển thị JSON dump ra UI). Hàm `getSuggestionsFromApi` có thể gọi không kèm `dishType` để nhận deck mixed.
- Định vị: `expo-location` lấy toạ độ user; fallback toạ độ trung tâm TP. HCM khi user từ chối quyền.
- Base URL backend cấu hình qua biến môi trường `EXPO_PUBLIC_API_BASE_URL`.

Để hỗ trợ pnpm monorepo (Metro bundler cần biết về workspace packages), `apps/mobile/metro.config.js` thêm `watchFolders`, `nodeModulesPaths`, bật `unstable_enableSymlinks` và `unstable_enablePackageExports`, đồng thời block `apps/backend/dist` khỏi vùng watch.

4.4 Giao diện web tham chiếu tại frontend

Thành phần web tham chiếu dùng Vite + React. Phần này phù hợp làm landing/demo hoặc mở rộng dashboard quản trị, có thể triển khai riêng trên Vercel theo chiến lược ở mục 8. Toàn bộ trải nghiệm chính của đề tài vẫn chạy trên mobile.

4.5 Phát triển cục bộ và kiểm thử

4.5.1 Chạy hệ thống cục bộ bằng Docker Compose

Repo cung cấp `docker-compose.yml` chạy PostgreSQL 16 + backend trong container: `docker compose up -d --build`. Compose mặc định inject các biến `DATABASE_URL` (trỏ tới Postgres trong cùng network), `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN`.

Khi connect tới Aiven PostgreSQL (production hoặc dev share), driver `pg` được cấu hình `ssl: { rejectUnauthorized: false }` trong `PrismaService` để chấp nhận self-signed certificate chain của Aiven.

4.5.2 Quy trình migration

Schema khai báo trong `prisma/schema.prisma`. Migration tạo bằng `prisma migrate dev`, áp dụng production qua `prisma migrate deploy` (chạy trong Dockerfile của backend trước khi start service). Seed dữ liệu (16 món Việt mặc định) thực hiện bằng `pnpm prisma db seed`.

4.5.3 Kiểm thử backend

Backend dùng Jest cho unit/e2e. Các script trong `apps/backend/package.json`: `test`, `test:cov`, `test:e2e`.

5 Trải nghiệm người dùng

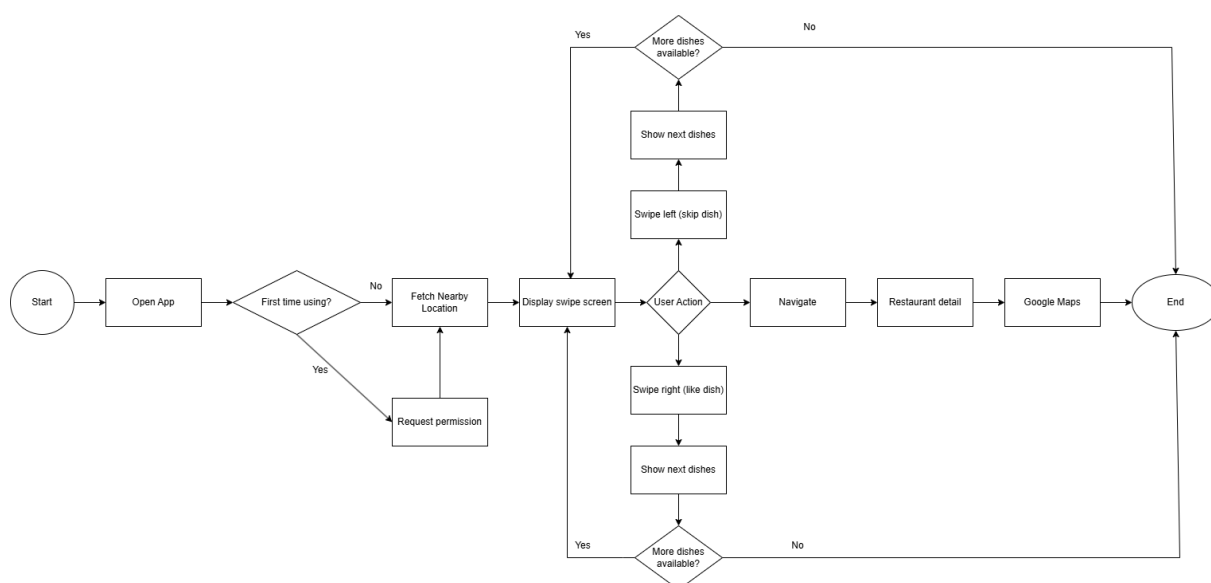
5.1 Luồng người dùng chi tiết

Luồng người dùng cho Sản phẩm khả thi tối thiểu (MVP) của Tinner được thiết kế trực quan và hấp dẫn, tập trung vào hai hành trình chính: khám phá các món ăn và quản lý các lựa chọn đã lưu.

5.1.1 Tính năng 1: Vuốt để khám phá và lưu món ăn

Tính năng này đóng vai trò là cơ chế khám phá chính, trình bày cho người dùng các tùy chọn thực phẩm gần đó tại một thời điểm để giảm tải nhận thức. Quy trình như sau:

1. **Khởi tạo:** Người dùng mở ứng dụng. Nếu đây là phiên đầu tiên của họ, hệ thống sẽ yêu cầu quyền truy cập vị trí để lấy các tùy chọn ăn uống gần đó.
2. **Màn hình khám phá:** Hệ thống hiển thị một thẻ món ăn bao gồm hình ảnh chất lượng cao, tên món ăn, giá cả, xếp hạng, khoảng cách và tên nhà hàng.
3. **Hành động của người dùng:** Người dùng đánh giá món ăn và có thể thực hiện một trong ba tương tác:
 - **Vuốt sang trái (Swipe Left):** Bỏ qua món ăn.
 - **Vuốt sang phải (Swipe Right):** Lưu món ăn vào danh sách "Muốn thử" cá nhân của họ.
 - **Nhấn Điều hướng (Tap Navigate):** Xem thông tin chi tiết về nhà hàng và mở Google Maps để nhận chỉ đường ngay lập tức.



Hình 1: Tính năng 1: Vuốt để khám phá và lưu món ăn

- Tính liên tục:** Sau một tương tác, hệ thống ngay lập tức trình bày món ăn tiếp theo có sẵn. Nếu không còn món ăn nào phù hợp với vị trí hoặc bộ lọc hiện tại, một thông báo "hết thẻ" sẽ được hiển thị.

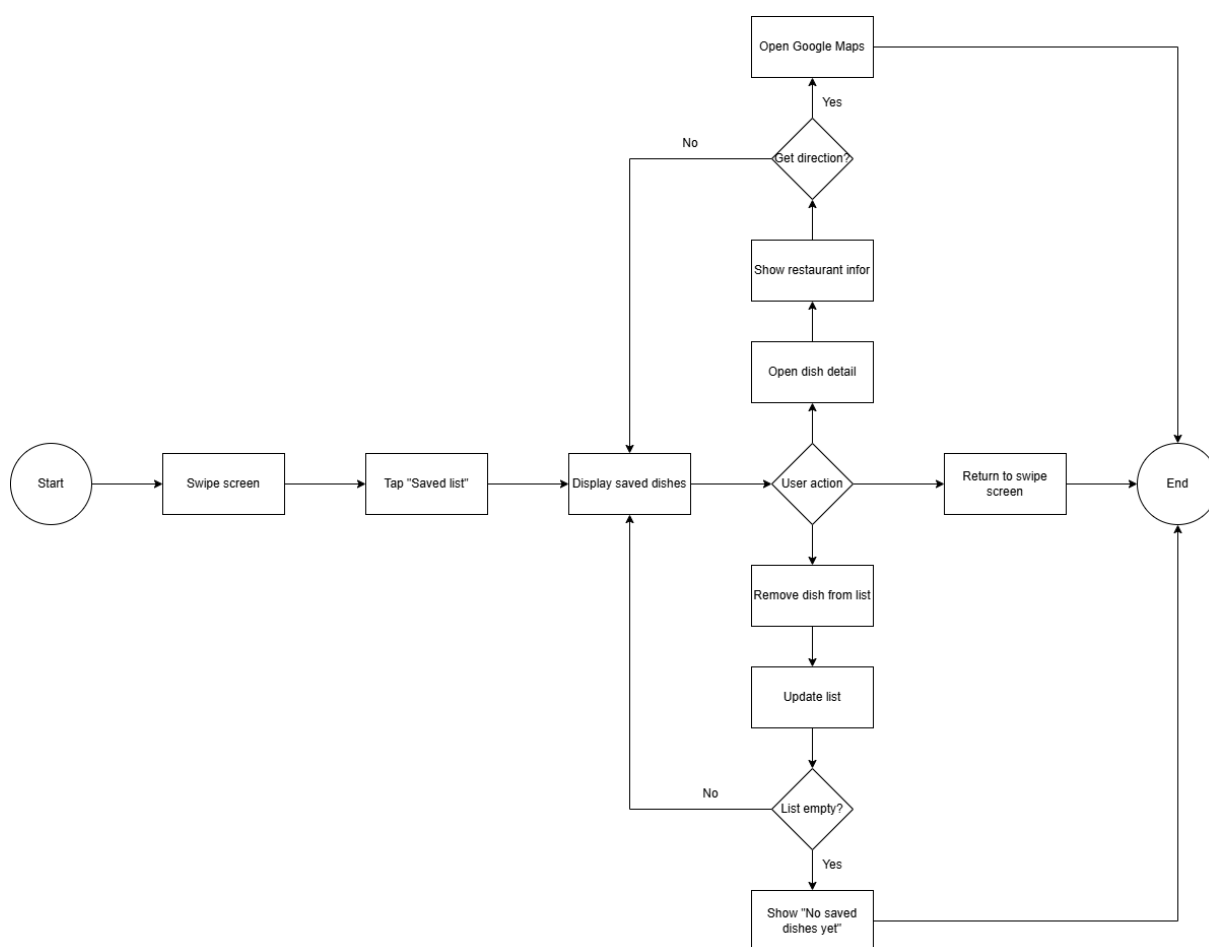
5.1.2 Tính năng 2: Quản lý danh sách “Muốn thử”

Tính năng này chuyển đổi giai đoạn khám phá ban đầu thành một quy trình ra quyết định có thể hành động bằng cách cho phép người dùng xem lại các mục họ đã thích.

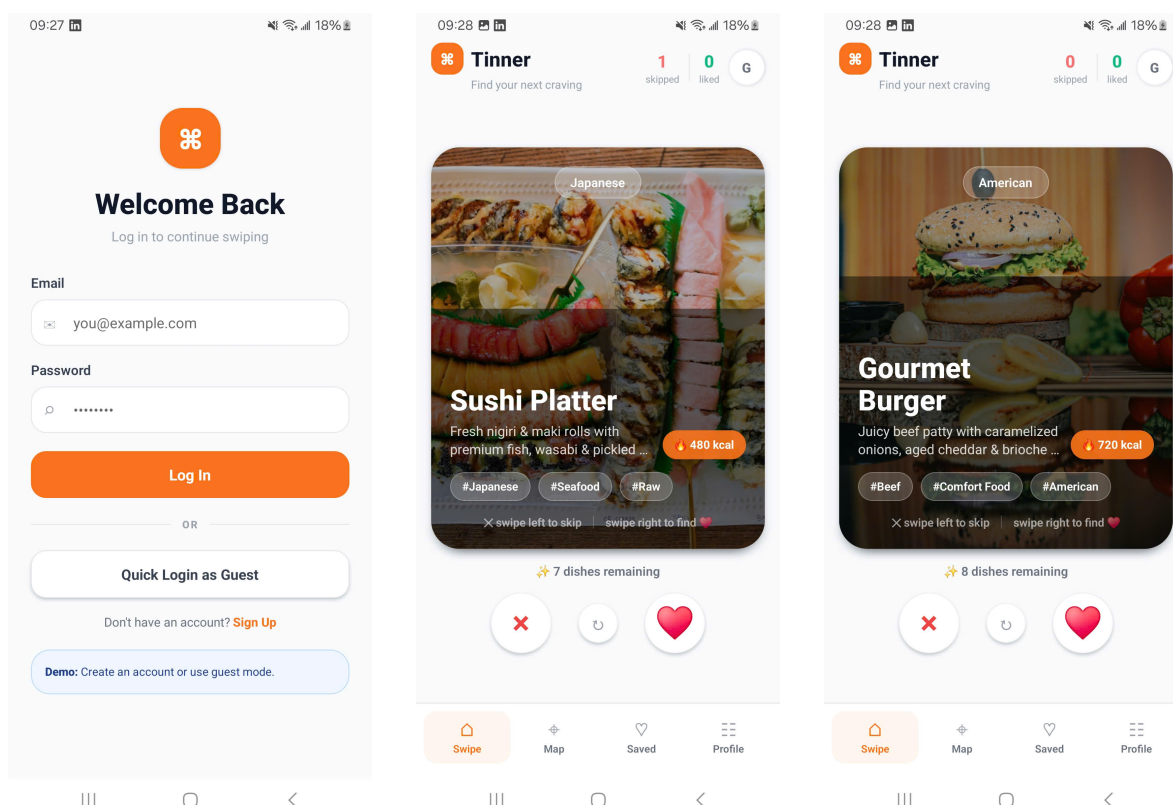
- Truy cập danh sách:** Người dùng điều hướng đến phần “Muốn thử” từ giao diện vuốt chính.
- Xem lại các tùy chọn:** Hệ thống trình bày một danh sách có cấu trúc gồm tất cả các món ăn đã lưu trước đó, cho phép so sánh trực tiếp.
- Hành động quyết định:** Người dùng có thể quản lý các lựa chọn của mình bằng cách:
 - Xem chi tiết:** Truy cập màn hình chi tiết món ăn và nhà hàng để nhận chỉ đường thông qua các dịch vụ bản đồ.
 - Xóa mục:** Xóa một món ăn khỏi danh sách nếu họ không còn quan tâm. Hệ thống xử lý các trạng thái danh sách trống một cách mượt mà nếu tất cả các mục bị xóa.
 - Quay lại:** Điều hướng quay lại màn hình vuốt chính để tiếp tục khám phá món ăn mới.

5.2 Ảnh chụp màn hình

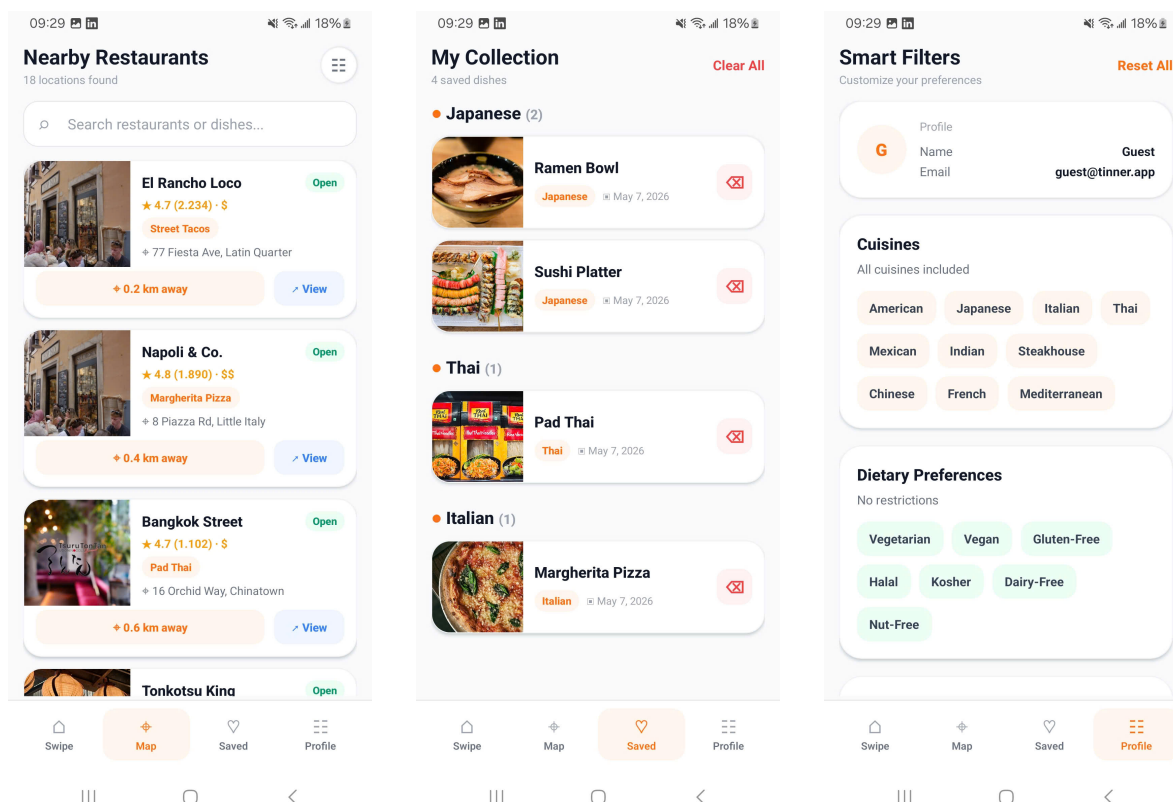
Các ảnh chụp màn hình sau đây minh họa các tương tác điều hướng và người dùng cốt lõi trong ứng dụng, thể hiện triết lý "Ưu tiên hình ảnh" (Visual-First) và giao diện người dùng trực quan.



Hình 2: Tính năng 2: Quản lý danh sách “Muốn thử”



Hình 3: Từ trái sang phải: Màn hình đăng nhập, Giao diện khám phá (Vuốt) hiển thị thông tin món ăn và Tương tác vuốt.



Hình 4: Từ trái sang phải: Tích hợp Bản đồ & Điều hướng, Danh sách "Muốn thử"(Đã lưu) và Hồ sơ người dùng.

6 Sơ đồ kiến trúc hệ thống

6.1 Mô tả kiến trúc tổng quan

Kiến trúc hệ thống theo mô hình client – server, gồm 4 tầng và một tích hợp third-party duy nhất:

- **Client layer:** Ứng dụng di động Expo/React Native là client chính. Web frontend (Vite + React) đóng vai trò tham chiếu/landing tùy chọn.
- **API layer:** Backend NestJS expose REST API tại prefix `/api/v1/*`. Các module nghiệp vụ (Auth, Dishes, Filters, Interactions, Suggestions) tách trách nhiệm rõ ràng; một `HealthController` riêng phục vụ giám sát.
- **Data layer:** PostgreSQL trên Aiven (production) hoặc Docker (local). Prisma quản lý schema, migration, truy vấn. DB còn đóng vai trò cache vĩnh viễn cho dữ liệu Overpass (xem chiến lược DB-first ở mục 4.2.4).
- **Third-party:** Backend tích hợp **Overpass API** (OpenStreetMap) để truy vấn nhà hàng theo tag `amenity` trong bán kính 7 km quanh user. Không yêu cầu API key, chỉ cần header `User-Agent`. `placeUrl` của mỗi nhà hàng là một URL Google Maps tĩnh dạng `https://www.google.com/maps/search/?api=1&query=lat,lng` — chỉ để client mở navigation, không gọi Google Maps API thực sự.
- **Quản trị phiên:** JWT access token + refresh token (lưu hash trong DB) để duy trì phiên và rotate.

6.2 Sơ đồ

Sơ đồ dưới đây mô tả luồng dữ liệu và quan hệ giữa các thành phần.

```
flowchart TB
    subgraph Clients
        Mobile[Expo React Native app-native]
        Web[Vite React Frontend]
    end

    subgraph Backend[NestJS Backend on Render]
        API[REST API /api/v1/*]
        Guard[JWT Auth Guard + GlobalExceptionHandler]
        Auth[Auth Module]
        Domain[Dishes / Filters / Interactions]
        Suggest[Suggestions Module - DB-first cache]
        Prisma[Prisma ORM]
    end

    DB[(PostgreSQL on Aiven)]
    Overpass[Overpass API OpenStreetMap]

    Mobile --> API
    Web --> API
    API --> Guard
    Guard --> Auth
    Guard --> Domain
    Guard --> Suggest
```

```
25 Auth --> Prisma
26 Domain --> Prisma
27 Suggest --> Prisma
28 Prisma --> DB
29 Suggest -.cache miss.-> Overpass
```

6.3 Luồng yêu cầu gợi ý nhà hàng

Luồng GET /api/v1/suggestions minh họa cách backend kết hợp DB-first cache với Overpass:

1. Mobile gửi request kèm lat, lng, có thể có dishType.
2. Backend đọc user_filters để biết maxDistanceKm (mặc định 5, cap 7) và minRating.
3. Suggestions service kiểm tra số nhà hàng đã sync trong bbox quanh user. Đủ ngưỡng → bỏ qua Overpass.
4. Nếu thiếu, gọi Overpass một lần (bán kính 7 km, filter amenity), parse cuisine/name để gán dishTypes, bulk insert qua createMany skipDuplicates.
5. Đọc DB cuối cùng: filter bbox + Haversine, sort theo distance, slice top N, shuffle nếu là chế độ mixed.

7 Thiết kế dịch vụ web

7.1 Kiểu dịch vụ và quy ước định tuyến

Backend cung cấp **REST API** với prefix versioning /api/v1. Bảng tổng hợp các endpoint hiện hữu:

Endpoint	Method	Mô tả ngắn
/auth/register	POST	Public. Đăng ký user mới, trả về tokens.
/auth/login	POST	Public. Đăng nhập bằng email/username + password.
/auth/refresh	POST	Public. Cấp lại access token từ refresh token.
/auth/logout	POST	Thu hồi refresh token hiện tại.
/auth/me	GET	Trả thông tin user đang đăng nhập.
/dishes	GET	Public. Trả danh sách 16 món Việt + image/description.
/filters	GET/PUT	Đọc/cập nhật bộ filter của user.
/interactions/like	POST	Lưu LIKE_DISH cho user.
/interactions/save	POST	Lưu SAVE_RESTAURANT khi swipe right.
/interactions/saved	GET	Danh sách quán đã lưu.
/interactions/saved/:id	DELETE	Bỏ lưu quán theo id interaction.
/suggestions	GET	Trả deck swipe theo lat, lng, dishType?
/health	GET	Public. Trạng thái backend ({status: "ok"}).

Toàn bộ endpoint trên đều có prefix /api/v1.

7.2 Xác thực và Phân quyền

Hệ thống dùng JWT 2 tầng:

- **Access token** (HS256, mặc định TTL 15 phút) cho mọi request cần xác thực. Mobile lưu trong memory state, gửi qua header `Authorization: Bearer <token>`.
- **Refresh token** (mặc định TTL 30 ngày) được sinh ngẫu nhiên, lưu hash trong bảng `refresh_tokens`. Khi access hết hạn, client gọi `/auth/refresh` để xoay vòng.

`JwtAuthGuard` đăng ký toàn cục qua `APP_GUARD`, áp dụng cho mọi controller mặc định. Để mở public, dùng decorator `@Public()` (set metadata, guard đọc và bỏ qua) — cấp class hoặc cấp method.

Luồng logout: client gửi refresh token hiện tại tới `/auth/logout`, backend match hash, xóa record trong DB. Sau đó access token cũ tự hết hạn theo TTL.

7.3 Validation, Xử lý lỗi và Bảo mật

- **Validation đầu vào**: DTO kèm `class-validator`, áp dụng pipeline global `ValidationPipe` với `transform: true` (cast string → number/enum), `whitelist` (strip field thừa), `forbidNonWhitelisted` (reject nếu có field lạ).
- **Xử lý lỗi thống nhất**: `GlobalExceptionHandler` (`@Catch()`) là middleware bắt tất cả exception. Phân biệt:

- `HttpException`: trích `status + message`, validation lỗi → `message = "Validation failed"`.
- Lỗi không lường (`Prisma`, `TypeError`, `network...`): log full stack server-side qua `Logger`, trả response generic `500 Internal server error` để không lộ chi tiết nội bộ.

Trường `details` (mảng lỗi validation cụ thể) chỉ kèm vào response khi `NODE_ENV != "production"` để hỗ trợ developer debug; production luôn giữ message ngắn gọn.

- **Client error format**: mobile client trong `api.ts` có hàm `extractErrorMessage` parse JSON response và lấy chỉ `.message` (fallback "Request failed with status N" nếu body không hợp lệ), tránh hiển thị JSON dump lên UI.
- **CORS**: bật `credentials` để hỗ trợ cookie/credentials cross-origin từ FE đã được trust.
- **Secrets qua biến môi trường**: `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN` chỉ inject ở runtime, không commit vào repo.
- **Password storage**: hash bằng `bcryptjs` (cost 12) trong `AuthService`.

7.4 Tài liệu hóa API

Backend tích hợp Swagger/OpenAPI qua `@nestjs/swagger` và `swagger-ui-express`. Tài liệu khởi tạo trong `main.ts` bằng `DocumentBuilder` với metadata (title, version, description) và `Bearer Auth` scheme. UI publish tại `/api/docs`.

Nhờ Swagger, nhóm có thể:

- Kiểm thử nhanh endpoint trong browser (paste access token vào ô Authorize).
- Đồng bộ contract giữa backend ↔ mobile mà không cần Postman riêng.
- Trong tương lai có thể export OpenAPI spec cho mock server hoặc generate typed client tự động.

Hướng nâng cấp tiếp:

- Bổ sung đầy đủ `@ApiTags`, `@ApiOperation`, `@ApiResponse` cho từng endpoint.
- Chuẩn hoá DTO với annotation Swagger để schema hiển thị chi tiết hơn.

8 Chiến lược triển khai

8.1 Mục tiêu triển khai

Chiến lược triển khai chia hệ thống thành các phần đặt ở những dịch vụ chuyên biệt, tận dụng managed service để giảm tải vận hành:

- **Database**: managed PostgreSQL trên Aiven.
- **Backend**: container web service trên Render.

- **Frontend web:** static/edge hosting trên Vercel.
- **Mobile distribution:** build và phân phối qua EAS (Expo Application Services).

Lưu ý: không tích hợp dịch vụ trả phí nào khác (không Google Maps API, không Foursquare). Toàn bộ dữ liệu nhà hàng đến từ Overpass API (OpenStreetMap), miễn phí và chỉ cần header User-Agent.

8.2 Công nghệ triển khai

- **Database:** PostgreSQL 16 trên **Aiven** (SSL bắt buộc, `sslmode=require`).
- **Backend: Render** (Docker web service, build từ `apps/backend/Dockerfile`).
- **FE web: Vercel** (Vite + React).
- **Mobile: EAS Build** (Expo) — tạo IPA/APK trở tới backend production qua `EXPO_PUBLIC_API_BASE_URL`.

8.3 Triển khai Backend trên Render

Repo có sẵn `render.yaml` cấu hình backend deploy theo Docker runtime:

- `type: web`
- `runtime: docker`
- `dockerfilePath: ./apps/backend/Dockerfile`
- `dockerContext: .` (lấy toàn monorepo, vì Dockerfile cần `packages/types`)
- `healthCheckPath: /api/v1/health`
- `autoDeploy: true`
- **biến môi trường:**
 - `NODE_ENV=production` (cố định) — đồng thời ẩn `validation details` khỏi response lỗi qua `GlobalExceptionHandler`.
 - `PORT=3000` (cố định)
 - `DATABASE_URL (sync: false)` — connection string Aiven có `sslmode=require`.
 - `JWT_ACCESS_SECRET, JWT_REFRESH_SECRET (sync: false)` — bí mật, sinh ngẫu nhiên.
 - `JWT_ACCESS_EXPIRES_IN=15m, JWT_REFRESH_EXPIRES_IN=30d`.

Dockerfile của backend chạy `prisma migrate deploy` ở entrypoint trước khi start app, đảm bảo schema luôn đồng bộ với code.

8.4 Chiến lược cơ sở dữ liệu trên Aiven

- Provision PostgreSQL 16 trên Aiven.
- Lấy connection string dạng `postgres://...?sslmode=require`, paste vào `DATABASE_URL` của Render.
- Trong code (`PrismaService`), driver pg cấu hình `ssl: { rejectUnauthorized: false }` để chấp nhận self-signed cert chain của Aiven mà không cần preload CA.
- Migration: `pnpm prisma migrate deploy` trong Docker endpoint.
- Seed (`prisma/seed.ts`) chạy thủ công khi cần: nạp 16 món Việt và bộ ảnh dishes.
- Backup: tận dụng tính năng PITR sẵn có của Aiven; không cần thao tác thêm trên ứng dụng.

8.5 Triển khai Frontend trên Vercel

Frontend web (tham chiếu) deploy lên Vercel:

- Import repo, đặt Root Directory tại `frontend`.
- Build command `pnpm build`; Output Directory theo mặc định Vite (`dist`).
- Biến môi trường: URL backend đã deploy trên Render (vd `VITE_API_BASE_URL`).
- Vercel tự xử lý preview deployment theo branch.

8.6 Phát hành ứng dụng di động qua EAS

Mobile build qua `eas build`:

- Cấu hình `eas.json` với profile `dev/preview/production`.
- Biến môi trường `EXPO_PUBLIC_API_BASE_URL` trỏ tới backend Render trong profile `production`.
- Build iOS/Android binary, phân phối qua Internal Distribution của EAS (TestFlight / Play Internal Track) hoặc qua expo dev client cho team test.

8.7 CI/CD và Vận hành

Render tự deploy mỗi khi push commit lên branch chính nhờ `autoDeploy: true`. Để tăng độ tin cậy, có thể bổ sung:

- CI trước merge (`lint + tsc -noEmit + Jest`) để chặn lỗi syntax/type rồi vào production.
- Monitoring/alert dựa trên health check `/api/v1/health` (Render có sẵn auto-restart khi health check fail).
- Tách môi trường `dev / staging / prod` với 3 Render service riêng + 3 Aiven DB riêng nếu cần.
- Tracking lỗi: cân nhắc thêm Sentry hoặc Render log streaming để gom log của `GlobalExceptionHandler`.

8.8 Tóm tắt

Chiến lược triển khai phù hợp MVP và vẫn mở rộng tốt:

- **Aiven** giảm rủi ro vận hành DB nhờ managed PITR + SSL.
- **Render** chạy backend NestJS dạng container, autoDeploy theo `render.yaml`, health check tự động.
- **Vercel** edge hosting cho frontend web tham chiếu.
- **EAS** build và phân phối mobile.
- Chi phí bên ngoài tối thiểu: không gọi API trả phí, chỉ dùng Overpass (miễn phí) cho dữ liệu nhà hàng.

9 Báo cáo độ bao phủ kiểm thử

9.1 Giới thiệu Jest và các công cụ testing

Trong dự án Tinner, chúng em sử dụng **Jest** làm khung thử nghiệm (testing framework) chính cho phần Backend. Jest là một công cụ mạnh mẽ được phát triển bởi Meta, được tối ưu hóa cho các ứng dụng JavaScript và TypeScript (thông qua `ts-jest`). Các lý do chính cho việc lựa chọn Jest bao gồm:

- **Tốc độ và hiệu suất:** Jest có khả năng chạy các bài kiểm tra song song, giúp giảm thiểu thời gian thực hiện toàn bộ suite test.
- **Mocking mạnh mẽ:** Hỗ trợ tạo các đối tượng giả (mocking) một cách dễ dàng, giúp cô lập các module cần kiểm tra (ví dụ: giả lập Database để thực hiện Unit Test).
- **Tích hợp sẵn Coverage:** Jest tích hợp thư viện Istanbul để tự động tính toán và báo cáo tỷ lệ bao phủ mã nguồn (code coverage) mà không cần cài đặt thêm công cụ bên ngoài.

Bên cạnh Jest, chúng em còn kết hợp với các công cụ bổ trợ khác như:

- **Supertest:** Sử dụng cho việc kiểm thử đầu cuối (E2E Testing), cho phép gửi các yêu cầu HTTP giả lập đến ứng dụng NestJS để kiểm tra luồng xử lý thực tế của API.
- **Prisma Mocking:** Sử dụng các thư viện giả lập để kiểm tra các Service mà không làm ảnh hưởng đến dữ liệu thực tế trong cơ sở dữ liệu PostgreSQL.

9.2 Cấu hình môi trường kiểm thử

Để đảm bảo các bài kiểm tra được thực hiện trong một môi trường ổn định và không ảnh hưởng đến dữ liệu thực tế, dự án đã thiết lập quy trình cấu hình như sau:

- **Tách biệt môi trường (Environment Isolation):** Sử dụng tệp cấu hình riêng (như `.env.test`) để định nghĩa các biến môi trường dành riêng cho việc kiểm thử, bao gồm mã khóa bí mật (JWT Secret) giả lập và đường dẫn cơ sở dữ liệu tạm thời.

- **Cấu hình Jest:** Các thông số kỹ thuật của Jest được định nghĩa trong tệp `jest.config.ts` hoặc trong tệp `package.json`. Tại đây, chúng em cấu hình `moduleNameMapper` để xử lý các đường dẫn tuyệt đối (aliases) và `testEnvironment` được thiết lập là `node` cho Backend.
- **Tự động hóa qua Scripts:** Các lệnh kiểm thử được tích hợp sẵn vào quy trình quản lý gói của dự án, cho phép thực hiện nhanh chóng qua các lệnh:
 - `pnpm test`: Chạy toàn bộ các bài Unit Test.
 - `pnpm test:cov`: Thực hiện kiểm thử và tự động xuất báo cáo độ bao phủ mã nguồn dưới dạng HTML và văn bản.
- **Xử lý Cơ sở dữ liệu:** Đối với các bài kiểm tra tích hợp (Integration Tests), chúng em sử dụng giải pháp **Database Mocking** hoặc kết nối với một container PostgreSQL tạm thời trong Docker để đảm bảo tính toàn vẹn của dữ liệu gốc.

9.3 Tiêu chuẩn độ bao phủ

Để đảm bảo chất lượng phần mềm và hạn chế tối đa lỗi phát sinh, nhóm chúng em đã đặt ra các tiêu chuẩn cụ thể về độ bao phủ mã nguồn (Code Coverage) như sau:

- **Mục tiêu tổng thể:** Đạt ít nhất **70%** độ bao phủ trên toàn bộ mã nguồn của hệ thống.
- **Các module cốt lõi (Core Modules):** Đối với các phần quan trọng như *Auth Service*, *Suggestions Service* và *Interactions Service*, mục tiêu độ bao phủ được đặt ra là trên **85%** để đảm bảo các luồng xử lý nghiệp vụ chính không bị sai sót.
- **Các chỉ số đánh giá:**
 - **Statement Coverage:** Đảm bảo hầu hết các dòng lệnh đều được thực thi ít nhất một lần.
 - **Branch Coverage:** Kiểm tra đầy đủ các điều kiện rẽ nhánh (`if/else`, `switch/case`) để tránh bỏ sót các trường hợp ngoại lệ.
 - **Function Coverage:** Đảm bảo tất cả các hàm và phương thức quan trọng đều được gọi trong quá trình kiểm thử.

9.4 Kết quả kiểm thử chi tiết

Dưới đây là bảng tổng hợp độ bao phủ mã nguồn (Code Coverage) cho các module chính trong thư mục `src` của Backend:

Bảng 2: Bảng tổng hợp Code Coverage theo Module

Module	% Stmts	% Branch	% Funcs	% Lines
Toàn bộ hệ thống	97.44	79.68	98.03	97.33
src/auth (Xác thực)	100.00	80.64	100.00	100.00
src/suggestions (Gợi ý)	93.10	85.36	93.10	93.12
src/interactions (Tương tác)	100.00	81.81	100.00	100.00
src/filters (Bộ lọc)	100.00	78.57	100.00	100.00
src/dishes (Món ăn)	100.00	75.00	100.00	100.00

```
Test Suites: 29 passed, 29 total
Tests:      95 passed, 95 total
Snapshots:  0 total
Time:       4.654 s
Ran all test suites.
PS D:\BK_semester6\Mobile\Tinner\apps\backend>
```

Hình 5: Kết quả lệnh kiểm tra Code Coverage trên Terminal

9.4.1 Module Auth (Xác thực)

Bảng dưới đây thể hiện độ bao phủ của các thành phần trong hệ thống xác thực người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
auth.controller.ts	100.00	75.00	100.00	100.00
auth.service.ts	100.00	84.21	100.00	100.00
jwt.strategy.ts	100.00	75.00	100.00	100.00

9.4.2 Module Gợi ý món ăn

Module này chịu trách nhiệm tìm kiếm nhà hàng từ Overpass API và Foursquare, đây là phần có logic phức tạp nhất:

File	% Stmts	% Branch	% Funcs	% Lines
suggestions.service.ts	80.64	70.00	87.50	80.70
overpass.service.ts	100.00	94.73	100.00	100.00
foursquare.service.ts	100.00	87.50	100.00	100.00
suggestions.controller.ts	100.00	75.00	100.00	100.00

9.4.3 Module Tương tác người dùng

Kiểm thử các hành động Like/Dislike món ăn và lưu trữ nhà hàng:

File	% Stmts	% Branch	% Funcs	% Lines
interactions.service.ts	100.00	90.00	100.00	100.00
interactions.controller.ts	100.00	75.00	100.00	100.00

9.4.4 Module Bộ lọc và Món ăn

Độ bao phủ của các module quản lý danh mục món ăn và bộ lọc người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
filters.service.ts	100.00	83.33	100.00	100.00
filters.controller.ts	100.00	75.00	100.00	100.00
dishes.service.ts	100.00	75.00	100.00	100.00

9.5 Các kịch bản kiểm thử API

Để đảm bảo tính đúng đắn của các nghiệp vụ, chúng em đã thực hiện kiểm thử trên các kịch bản quan trọng sau:

1. Xác thực (Authentication):

- Đăng ký tài khoản mới với đầy đủ thông tin hợp lệ (Thành công).
- Đăng nhập với email và mật khẩu chính xác (Trả về Access Token).
- Đăng nhập với mật khẩu sai (Trả về lỗi 401 Unauthorized).
- Truy cập vào các API bảo mật (như Profile) khi không có Token (Bị từ chối).

2. Gợi ý món ăn (Suggestions):

- Lấy danh sách gợi ý khi cung cấp đầy đủ tọa độ lat và lng (Thành công).
- Yêu cầu gợi ý nhưng thiếu tọa độ (Trả về lỗi 400 Bad Request do Validation).
- Lọc danh sách gợi ý theo loại món ăn (Ví dụ: chỉ lấy quán "Phở").

3. Tương tác người dùng (Interactions):

- Thực hiện hành động "Like" một món ăn (Lưu vào cơ sở dữ liệu).
- Thực hiện lưu một nhà hàng vào danh sách yêu thích (Thành công).

4. Bộ lọc (Filters):

- Cập nhật khoảng cách tìm kiếm tối đa và đánh giá tối thiểu (Thành công).
- Kiểm tra xem các gợi ý sau khi cập nhật có tuân thủ đúng bộ lọc mới hay không.

9.6 Phạm vi kiểm thử E2E Layer

Dưới đây là bảng tổng hợp các kịch bản kiểm thử End-to-End đã được thực hiện để kiểm chứng luồng hoạt động thực tế của hệ thống:

Bảng 3: Bảng tổng hợp kịch bản kiểm thử E2E

E2E Module	Số tests	Các scenarios được cover
auth.e2e.ts	5	register new user, login, get profile with token
suggestions.e2e.ts	4	fetch restaurants around, sync from Overpass
interactions.e2e.ts	3	like dish flow, save restaurant, check swiped history
dishes.e2e.ts	3	get all categories, list dishes by type
filters.e2e.ts	2	update and apply filters to results
app.e2e.ts	2	health check, root path check

```
PASS test/app.e2e-spec.ts
PASS test/filters.e2e-spec.ts
PASS test/suggestions.e2e-spec.ts
PASS test/dishes.e2e-spec.ts
PASS test/interactions.e2e-spec.ts
PASS test/auth.e2e-spec.ts

Test Suites: 6 passed, 6 total
Tests:       19 passed, 19 total
Snapshots:   0 total
Time:        2.269 s
Ran all test suites.
PS D:\BK_semester6\Mobile\Tinner\apps\backend>
```

Hình 6: Kết quả lệnh kiểm thử End-to-End trên Terminal

9.6.1 Phương pháp tiếp cận và Cấu trúc kiểm thử E2E

Khác với Unit Test chỉ tập trung vào một Service hay Controller đơn lẻ, kiểm thử E2E trong dự án được thiết kế để đánh giá sự tích hợp của toàn bộ chu trình xử lý request. Quá trình này được thực hiện thông qua các công cụ và phương pháp sau:

- **Mô phỏng HTTP Request bằng Supertest:** Thư viện `supertest` được sử dụng để giả lập các yêu cầu HTTP (GET, POST, PUT, DELETE) gửi trực tiếp đến ứng dụng NestJS. Điều này giúp kiểm chứng chính xác các mã lỗi HTTP (Status Codes), cấu trúc JSON trả về và các quy tắc xác thực dữ liệu đầu vào (Validation Pipes).
- **Cơ chế Subcutaneous Testing:** Thay vì kết nối với một cơ sở dữ liệu PostgreSQL thực tế, dự án áp dụng kỹ thuật giả lập (mocking) ở tầng Database. Thông qua `overrideProvider(PrismaService)`, các truy vấn cơ sở dữ liệu được kiểm soát hoàn toàn. Nhờ vậy, 19 kịch bản E2E có thể hoàn thành xuất sắc và ổn định chỉ trong khoảng 2 giây.
- **Giả lập xác thực (Mocking Guards):** Đối với các API yêu cầu quyền truy cập (Protected Routes), `JwtAuthGuard` được làm giả lập để cung cấp thông tin người dùng ảo (`req.user`). Việc này giúp kiểm thử trực tiếp các logic nghiệp vụ bên trong mà không cần thực hiện lại quy trình đăng nhập lấy Token cho mỗi test case riêng biệt.

9.6.2 Đánh giá kết quả kiểm thử E2E

Toàn bộ 19/19 kịch bản kiểm thử E2E phân bổ trên 6 Test Suites đều vượt qua thành công (100% Pass Rate). Qua đó, hệ thống chứng minh được:

- **Tính toàn vẹn của luồng dữ liệu:** Sự phối hợp giữa Routing, Validation DTO, Controller và Service hoạt động trơn tru. Các ngoại lệ (Exceptions) như trùng lặp

email khi đăng ký hay sai mật khẩu đều được tăng Exception Filter bắt lại và xử lý trả về đúng định dạng lỗi.

- **Độ tin cậy của API:** Các phản hồi HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found) đều được phản hồi chính xác theo đúng tài liệu thiết kế API.
- **Tối ưu quy trình CI/CD:** Nhờ kiến trúc test không phụ thuộc vào các dịch vụ bên thứ 3 (như Database hay Overpass API), bộ E2E test rất nhẹ và không bị "flaky" (lúc pass lúc fail). Điều này tạo tiền đề vững chắc để tích hợp vào các luồng kiểm tra tự động (CI/CD) trong tương lai.

9.7 Gap Analysis and Code Quality

10 Phân tích khảo sát mức độ hài lòng và Hành vi người dùng

10.1 Phản hồi và Kết quả khảo sát

10.2 Số liệu hành vi người dùng

11 Đánh giá dự án và Các hướng cải thiện

11.1 Thành công và Thách thức

11.2 Các kế hoạch trong tương lai

12 Sản phẩm thực tế

12.1 Khả năng tương thích của ứng dụng

12.2 Tệp APK và Liên kết Google Drive

12.3 Tích hợp Backend và Cấu hình

13 Phân phối sản phẩm và Khảo sát

13.1 Quảng bá và Tiếp thị

13.2 Thiết kế và Phân tích khảo sát

13.3 Thu thập số liệu ứng dụng

13.4 Đánh giá phản hồi và Phân tích hành vi

13.5 Minh chứng tương tác người dùng